

Docent: Dirk Hostens

Apps for Android

Opdracht: Introductie in Jetpack Compose

Bronnen:

Zie Toledo



In een reeks van opdrachten maken we een app om een lijst van todo items te beheren. We zullen taken aanmaken en toekennen aan gebruikers en uiteraard ook een gebruikersbeheer voorzien. Elke opdracht zal verder bouwen op de app van de vorige opdracht.

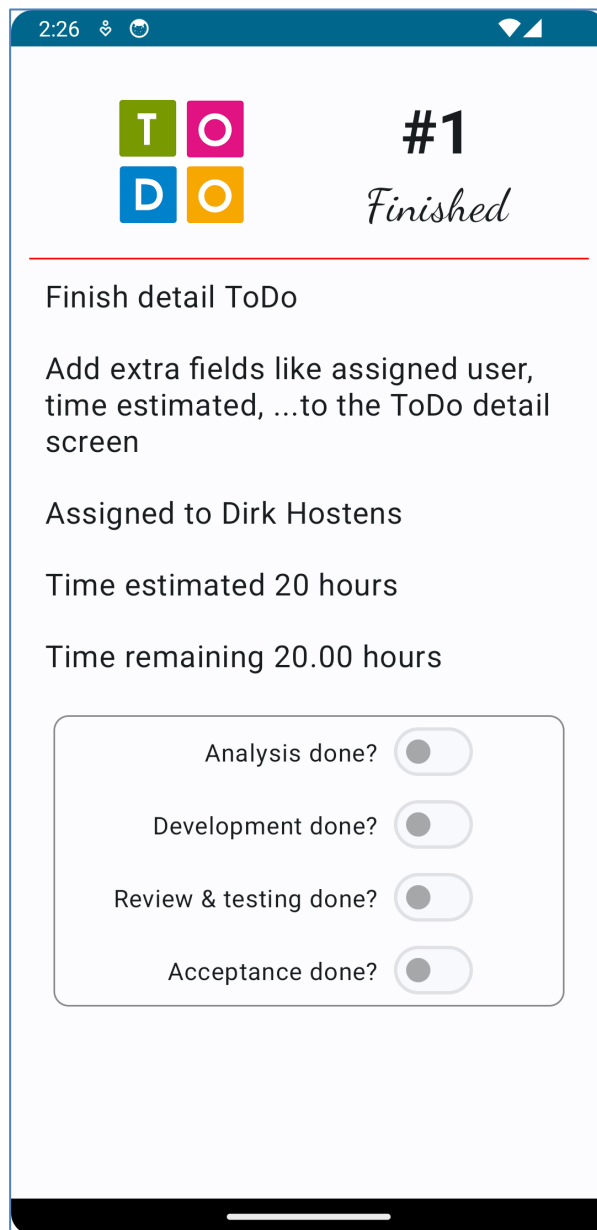
Na deze opdracht kan je een basis layout aanmaken via Jetpack composable's en begrijp je ook de achterliggende mechanismen als recomposition, annotations, het verschil tussen een Composable function en een Kotlin function, het gebruik van resources in je project, ...

In deze eerste opdracht is het belangrijk om zelf te coderen en enkel gebruik te maken van Gemini om 'boiler plate' code te laten genereren. Naar mate de opdrachten vorderen maken we meer en meer gebruik van Gemini AI.

In deze opdracht

- maak je een Android app project aan in Android Studio
- programmeer je een eerste scherm layout met behulp van Jetpack Compose en de tool Gemini
- definieer je eigen Jetpack composables functies gebruikmakend van Text() en Switch()
- leer je het gebruik van containers als Column en Row
- kan je een afbeelding importeren in het project en integreren in de UI

Het eerste scherm ziet er als volgt uit.



Je maakt een nieuw Empty Activity project aan waarbij je de package name als volgt definieert: be.**voornaamfamilienaam**.appnaam

In de app zullen we gebruik maken van de models User en ToDo. De eerste stap bestaat er dan ook in om de nodige klassen aan te maken in de package models.

Een ToDo object heeft volgende velden en constructors:

- number: Int *
- title: String *
- description: String *
- createdByUser: User *
- createOnDate: Date *
- assignedToUser: User? *
- finishedOnDate: Date? *
- timeEstimated: Int *
- analysisDone : Boolean
- developmentDone: Boolean
- reviewAndTestingDone: Boolean
- acceptanceDone : Boolean

Daarnaast voorzie je nog een aantal berekende velden.

- status
 - o wordt bepaald door de velden assignedTo en finishOnDate
 - o we maken hiervoor gebruik van een enum Status met mogelijke waarden NEW, ASSIGNED en FINISHED.
 - FINISHED -> assignedToUser != null && finishedOnDate != null
 - ASSIGNED -> s
 - In het ander geval NEW

```
enum class Status {  
    NEW, ASSIGNED, FINISHED  
}
```

```
val status : Status  
get() {  
    return if (assignedToUser != null && finishedOnDate != null) {  
        Status.FINISHED  
    } else if (assignedToUser != null) {  
        Status.ASSIGNED  
    } else {  
        Status.NEW  
    }  
}
```

- statusDescription – String waarde op basis van status
- timeRemaining van type Int
 - o Dit wordt berekend op basis van de Boolean variabelen analysisDone, developmentDone, reviewAndTestingDone & acceptanceDone
 - Bij analysisDone: timeRemaining = 85% van timeEstimated
 - Bij developmentDone: timeRemaining = 30% van timeEstimated
 - Bij a timeRemaining = 10% van timeEstimated
 - Bij acceptanceDone: timeRemaining = 0

Een User object heeft volgende velden en constructors:

- id: Int
- userName: String
- firstName: String
- lastName: String
- password: String
- isActive: Boolean

Een constructor met de alle velden en een tweede empty constructor. **Gebruik de Gemini agent om dit te genereren.**

In latere fase zullen we de gebruikers en todo items ophalen uit een database maar om het scherm en de rest van de functionaliteit nu al uit te werken, voorzien we een functie die een list van User of Todo objecten teruggeeft. Voorzie 3 User objecten en 5 Todo's objecten (verschillende status). **Vraag aan Gemini agent hoe dit best te implementeren.**

Nu zijn we klaar om het scherm uit te werken. (zie pagina 2). Dit doen we in eerste fase zonder Gemini agent. Het doel van de cursus is leren hoe een Android app in Kotlin van scratch is opgebouwd. Het is belangrijk om in het begin zelf ervaring op te bouwen en niet alleen te vertrouwen op AI.

Maak een nieuwe Composable function ToDetailScreen. Linksboven vind je een afbeelding met daarnaast het nummer van de Todo en de status. Eronder is een rode lijn voorzien. De eerste 2 velden onder de lijn zijn de titel en description, gevolgd door time estimated en time remaining. Het onderste gedeelte is omlijnd en toont de waarden van de boolean variabelen in een Switch. .

Extra info:

Om in de preview alle Todo items te zien kan je volgende toevoegen:

```
//Preview data of ToDo
class UserPreviewParameterProvider : PreviewParameterProvider<ToDo> {
    private val todos = MockupToDo.getTodos()
    override val values = todos.asSequence()
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun GreetingPreview(@PreviewParameter(ToDoPreviewParameterProvider::class,
limit = 3) todo: ToDo) {
    MyToDoAppTheme {
        MyToDoLayout(todo)
    }
}
```